

Cloud Computing

Lecture 3

Virtualization Technology

Virtualization and cloud computing

- Plays an important role in cloud computing.
- Primarily used to offer configurable computing environments and storage.
- H/w virtualization enabling solution in IaaS
- Programming language virtualization in PaaS.
- Virtualization provides :-
 - Consolidating
 - Isolation
 - Controlled environments

Operating Systems Limitations

- OSs provide a way of virtualizing hardware resources among *processes*
- This may help isolate *processes* from one another
- However, this does not provide a virtual machine to a user who may wish to run a different OS
- Having hardware resources managed by a single OS limits the flexibility of the system in terms of available software, security, and failure isolation
- Virtualization typically provides a way of relaxing constraints and increasing flexibility

Virtualization Properties

- Fault Isolation
- Software Isolation
- Performance Isolation (accomplished through scheduling and resource allocation)

Isolation

1

- All VM state can be captured into a file (i.e., you can operate on VM by operating on file— cp, rm)
- Complexity is proportional to virtual HW model and independent of guest software configuration

Encapsulation

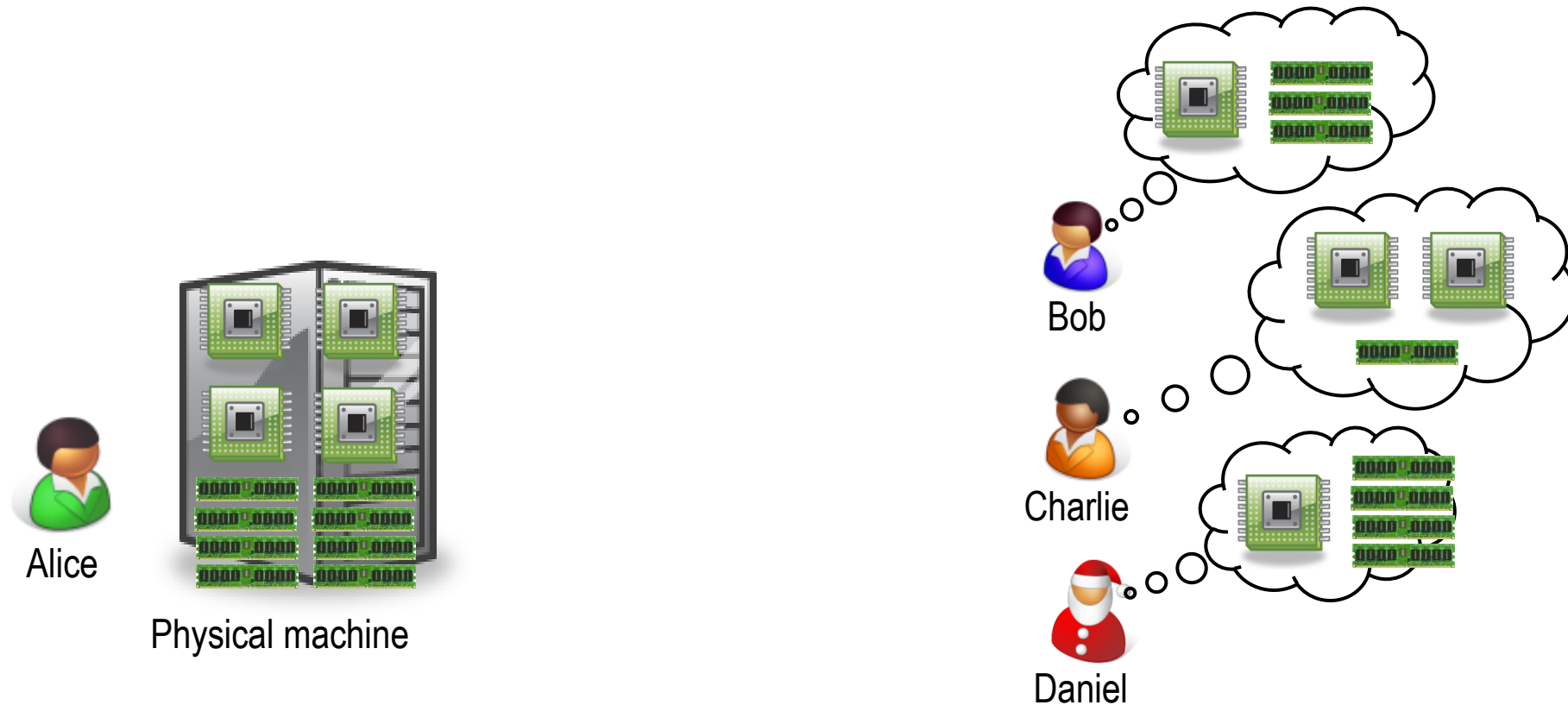
2

- All guest actions go through the virtualizing software which can inspect, modify, and deny operations

Interposition

3

What is virtualization?

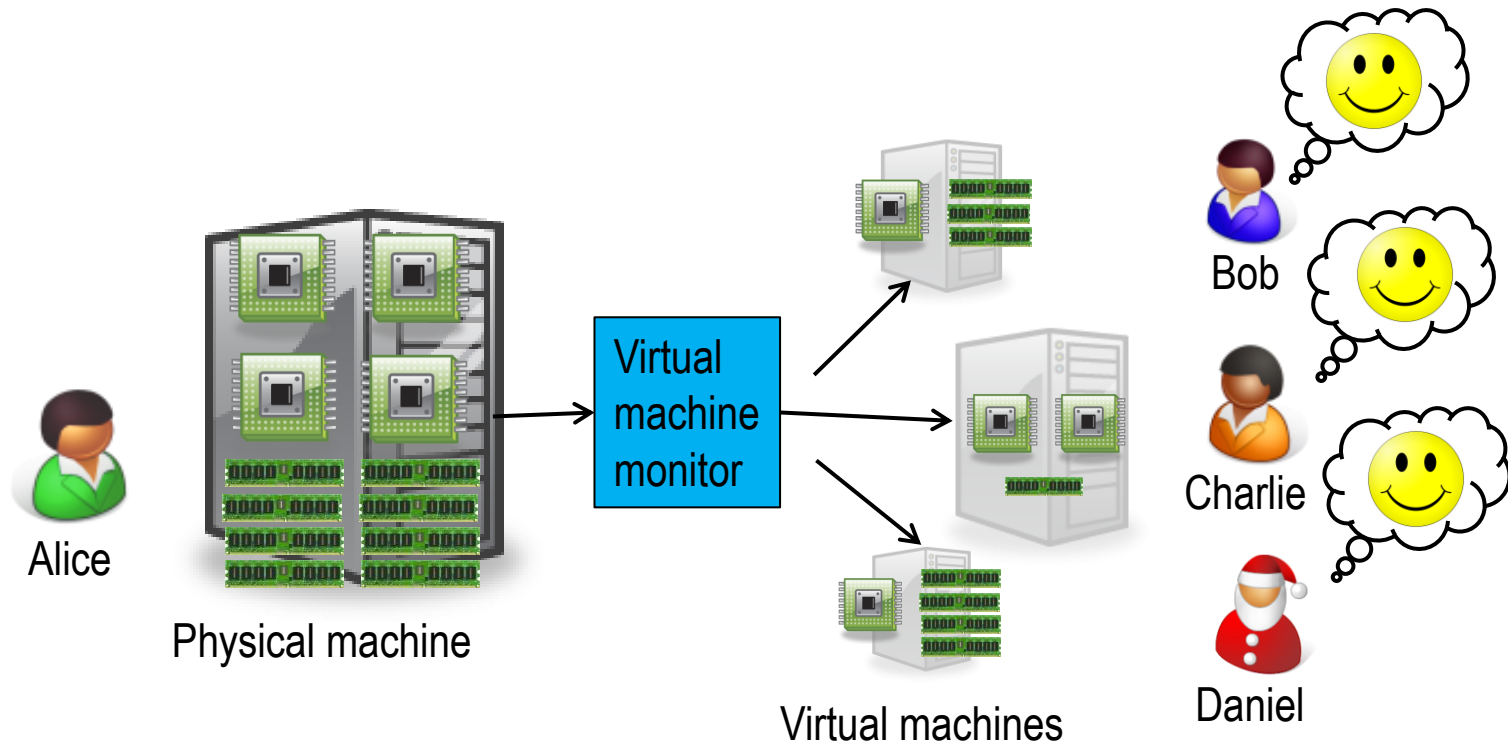


Suppose Alice has a machine with 4 CPUs and 8 GB of memory, and three customers:

- Bob wants a machine with 1 CPU and 3GB of memory
- Charlie wants 2 CPUs and 1GB of memory
- Daniel wants 1 CPU and 4GB of memory

What should Alice do?

What is virtualization?



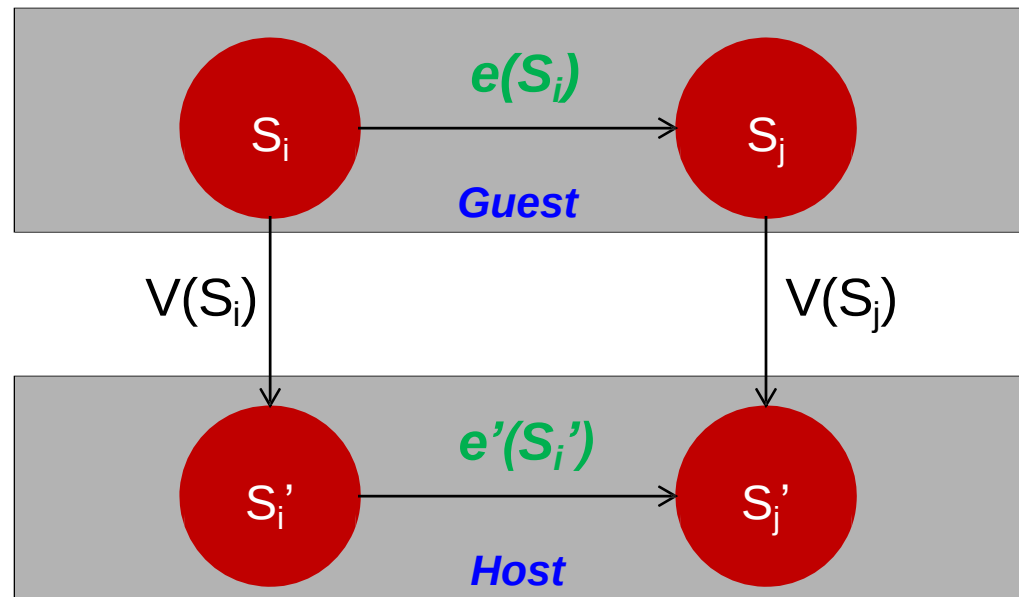
Alice can sell each customer a **virtual machine** (VM) with the requested resources

- From each customer's perspective, it appears as if they had a physical machine all by themselves (**isolation**)

What is Virtualization?

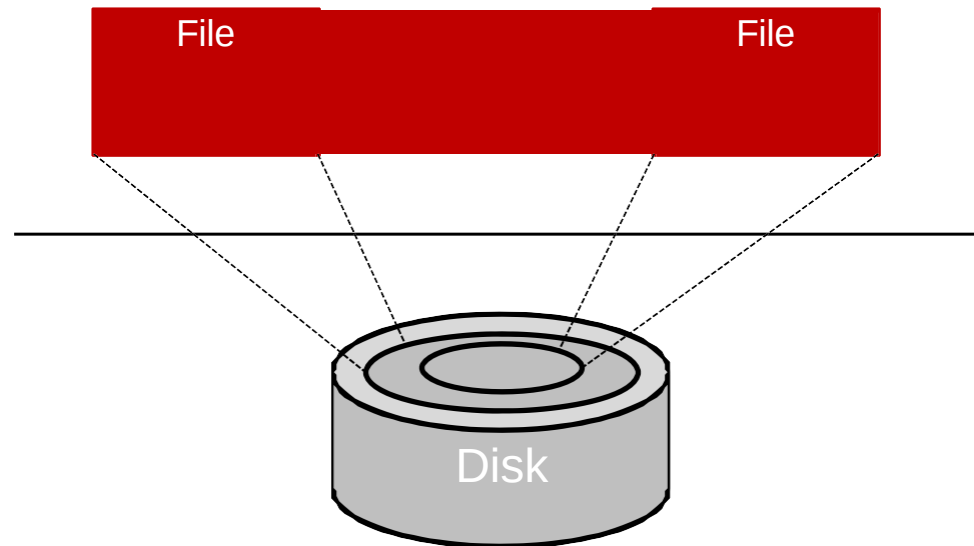
- Informally, a virtualized system (or subsystem) is a mapping of its interface, and all resources visible through that interface, to the interface and resources of a real system
- Formally, virtualization involves the construction of an isomorphism that maps a virtual *guest* system to a real *host* system (Popek and Goldberg 1974)

- ✓ Function V maps the guest state to the host state
- ✓ For a sequence of operations, e , that modifies a guest state, there is a corresponding e' in the host that performs an equivalent modification
- ✓ How can this be managed?



Abstraction

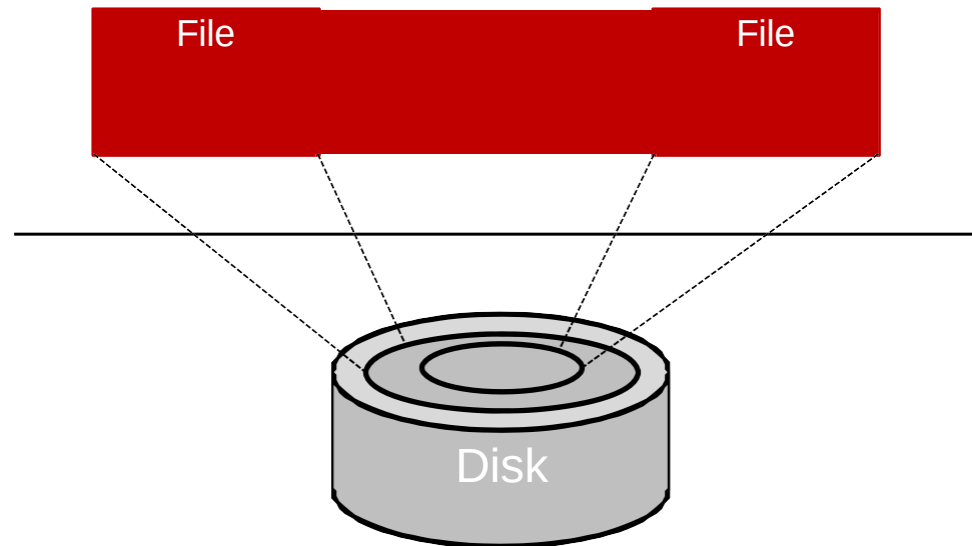
- The key to managing complexity in computer systems is their division into *levels of abstraction* separated by *well-defined interfaces*
- Levels of abstraction allow implementation details at lower levels of a design to be ignored or simplified



Abstraction

E.g. disk storage

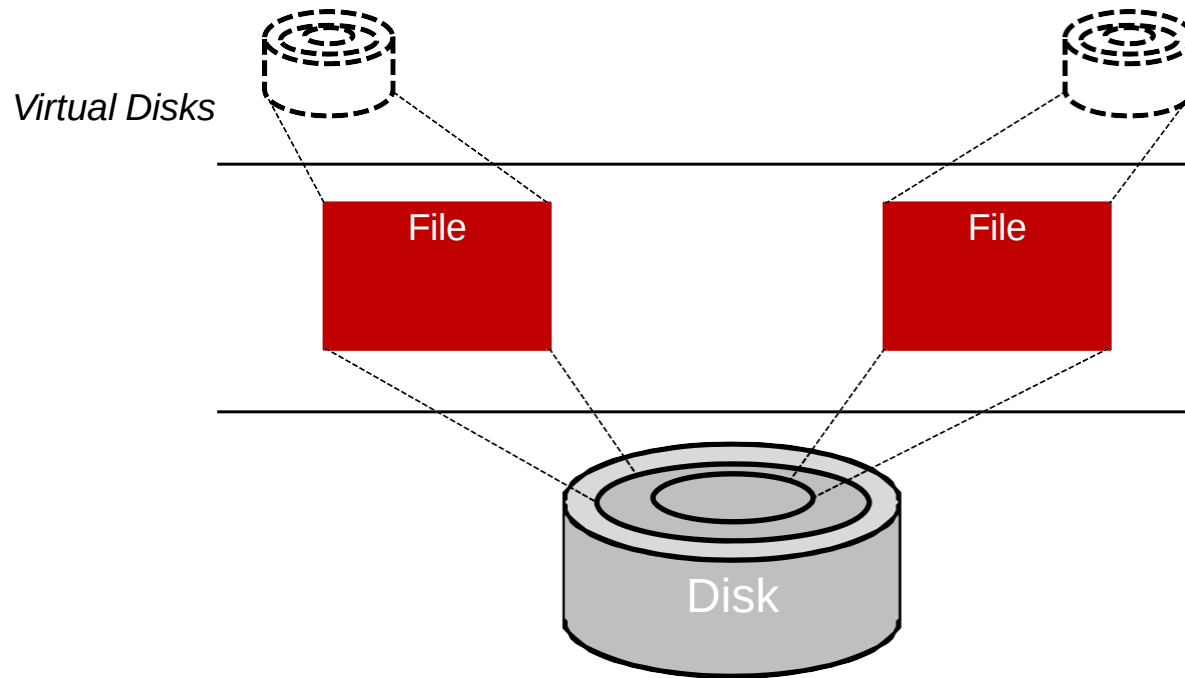
- Hides hard-disk addressing details (sectors and tracks)
- It appears to application software as a variable sized files.
- User can create, write and read files without knowing the underneath details.



- ✓ Files are an abstraction of a Disk
- ✓ A level of abstraction provides a simplified interface to underlying resources

Virtualization and Abstraction

- Virtualization uses abstraction but is different in that it doesn't necessarily hide details; the level of detail in a virtual system is often the same as that in the underlying real system



✓ *Virtualization provides a different interface and/or resources at the same level of abstraction*

Programming language-level virtualization

- It is mostly used to achieve *ease of deployment* of application, *managed execution* and *portability across* different platform and OS.
- It consists of a virtual machine *executing the byte code of a program*, which is the result of the *compilation process*.
- Produce a binary format representing the machine code for an abstract architecture.
- Example
 - Java platform – Java virtual machine (JVM)
 - .NET provides Common Language Infrastructure (CLI)
- They are stack-based virtual machines

Application-level virtualization

- It is a technique allowing applications to run in runtime environments that do not natively support all the features required by such applications.
- In this, applications are not installed in the expected runtime environment.
- This technique is most concerned with :-
 - Partial file system
 - Libraries
 - Operating System component emulation
- E.g. Wine, CrossOver...

Virtual Machines and Hypervisors

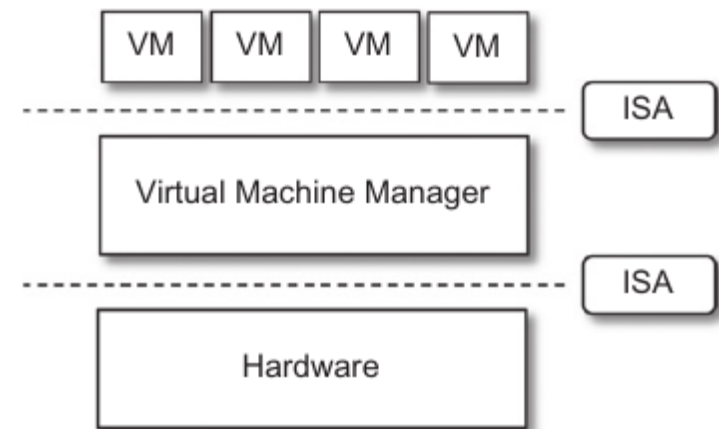
- The concept of virtualization can be applied not only to subsystems such as disks, but to an entire machine denoted as a **virtual machine (VM)**
- A VM is implemented by adding a layer of software to a real machine so as to support the desired VM's architecture
- This layer of software is often referred to as **virtual machine monitor (VMM)**
- Early VMMs are implemented in firmware
- Today, VMMs are often implemented as a co-designed firmware-software layer, referred to as the **hypervisor**

Hypervisor

- Hypervisor runs above the supervisor mode.
- It runs in supervisor mode.
- It recreates a h/w environment.
- It is a piece of s/w that enables us to run one or more VMs on a physical server(host).
- Two major types of hypervisor
 - *Type -I*
 - *Type-II*

Type-I Hypervisor

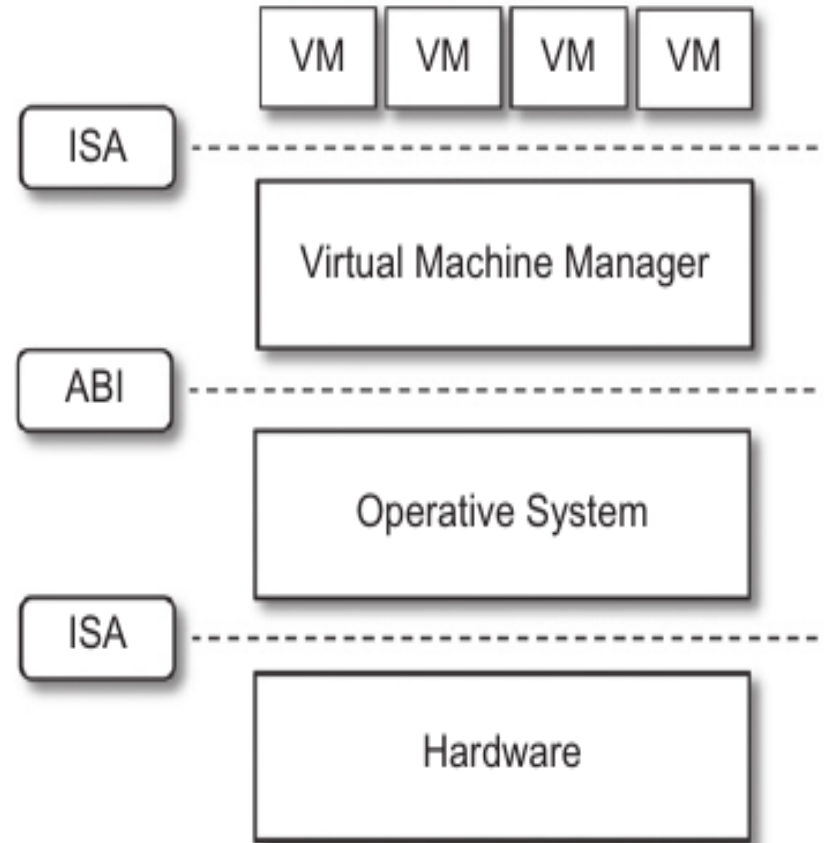
- It runs directly on top of the hardware.
- Takes place of OS.
- Directly interact with the ISA exposed by the underlying hardware.



- Also known as *native virtual machine*.

Type-II Hypervisor

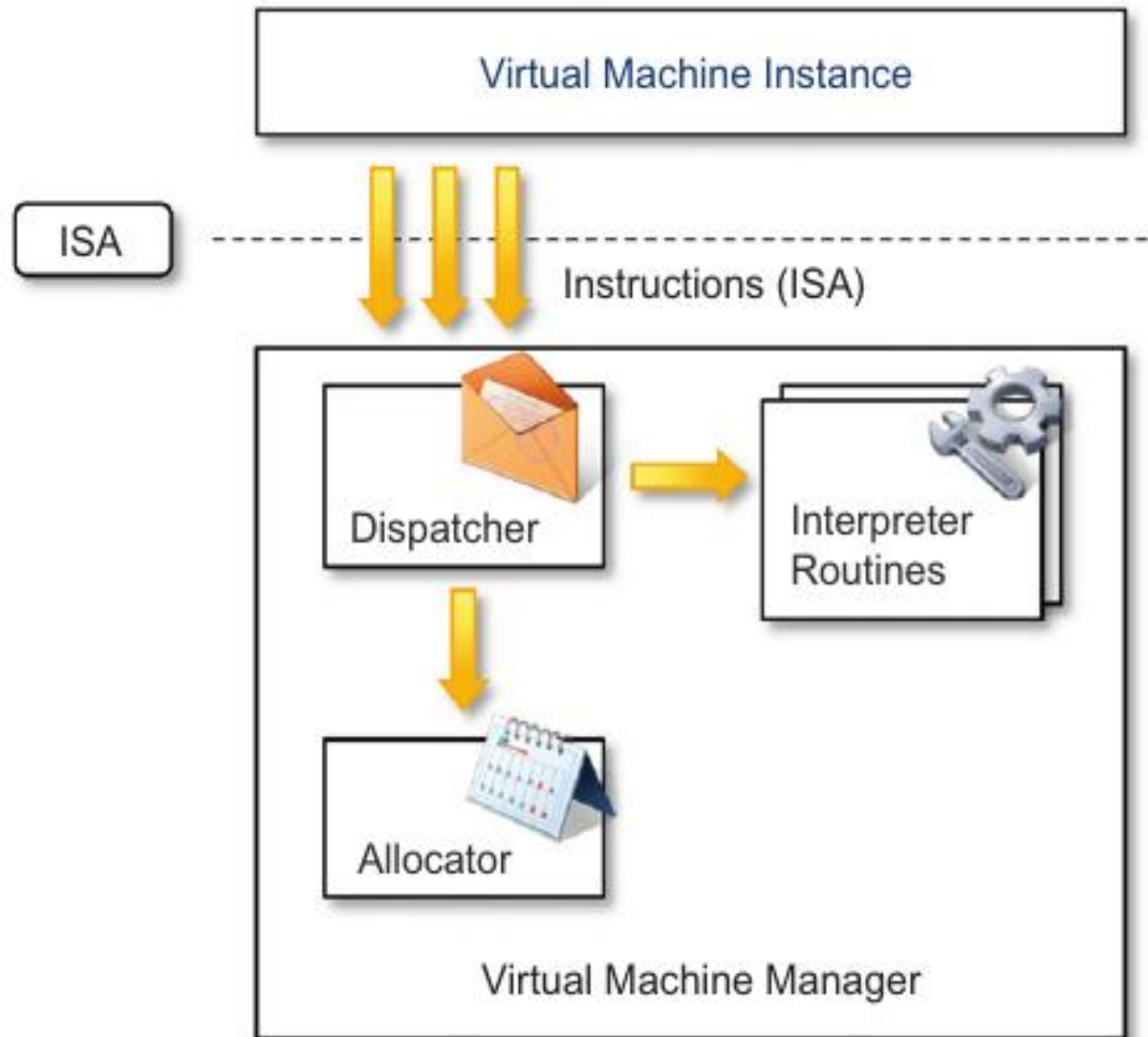
- It requires the support of an operating system to provide virtualization services.
- Programs managed by the OS.
- Emulate the ISA of virtual h/w.
- Also called hosted virtual machine.



Virtual Machine Manager (VMM)

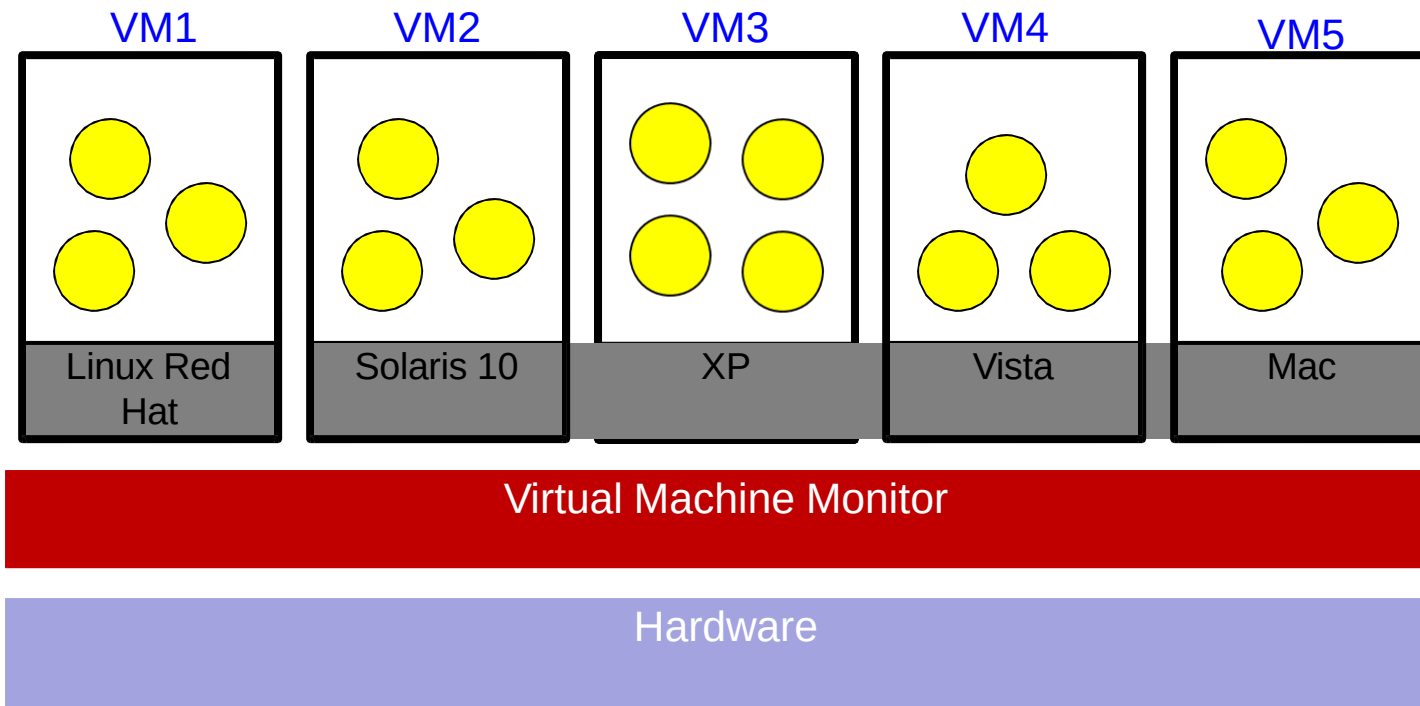
- Main Modules
 - **Dispatcher**
 - Entry Point of VMM
 - Reroutes the instructions issued by VM instance.
 - **Allocator**
 - Deciding the system resources to be provided to the VM.
 - Invoked by dispatcher
 - **Interpreter**
 - Consists of interpreter routines
 - Executed whenever a VM executes a privileged instruction.
 - Trap is triggered and the corresponding routine is executed.

Virtual Machine Manager (VMM)



A Mixed OS Environment

- Multiple VMs can be implemented on a single hardware platform to provide individuals or user groups with their own OS environments



Full Virtualization

- Traditional VMMs provide full-virtualization:
 - The functionality provided is identical to the underlying physical hardware
 - The functionality is exposed to the VMs
 - They allow unmodified guest OSs to execute on the VMs
 - This might result in some performance degradation
- E.g., *VMWare* provides full virtualization

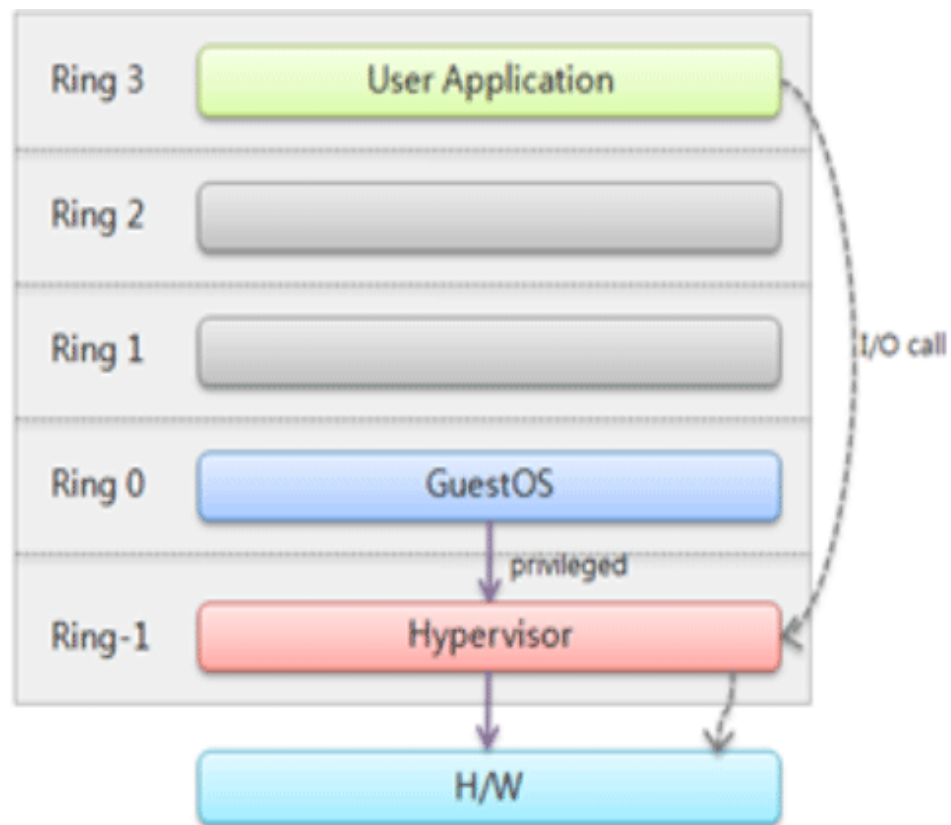
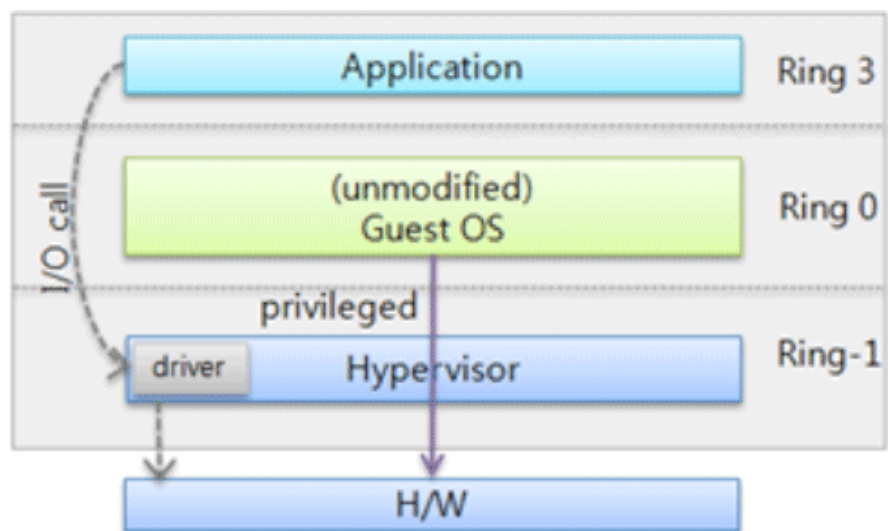
Full virtualization

- Advantages
 - Complete isolation
 - Enhanced security
 - Ease of emulation of different architectures and coexistence
- Key challenge
 - interception of privileged instructions

Full virtualization

– Hardware-assisted virtualization

- In this hardware provides architectural support for building a VMM able to run a guest OS in complete isolation.
- Intel VT and AMD V extensions.
- Early products were using binary translation to trap some sensitive instructions and provide an emulated version
- Additional Ring -1
- *No binary translation* of privileged instructions
- Commands are are executed directly to h/w via the hypervisor



Para-Virtualization

- Other types of VMMs provide **para-virtualization**:
 - They provide a virtual hardware abstraction that is similar, but not identical to the real hardware
 - They modify the guest OS to cooperate with the VMM
 - They result in lower overhead leading to better performance
 - E.g., *Xen* provides both para-virtualization as well as full-virtualization

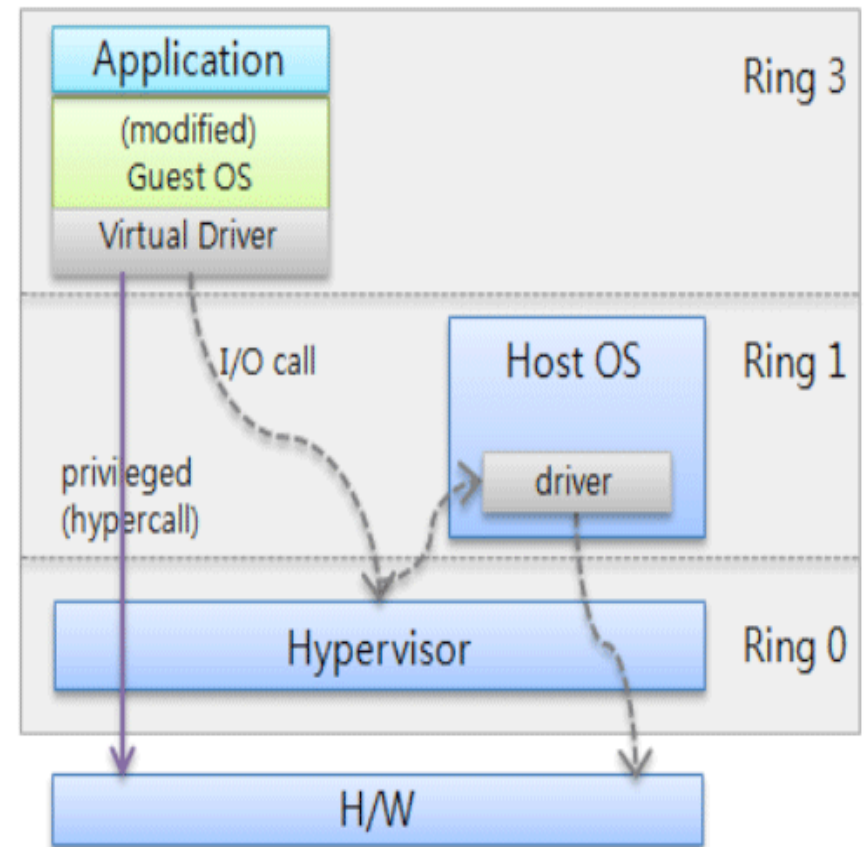
Para-Virtualization

Specifications:

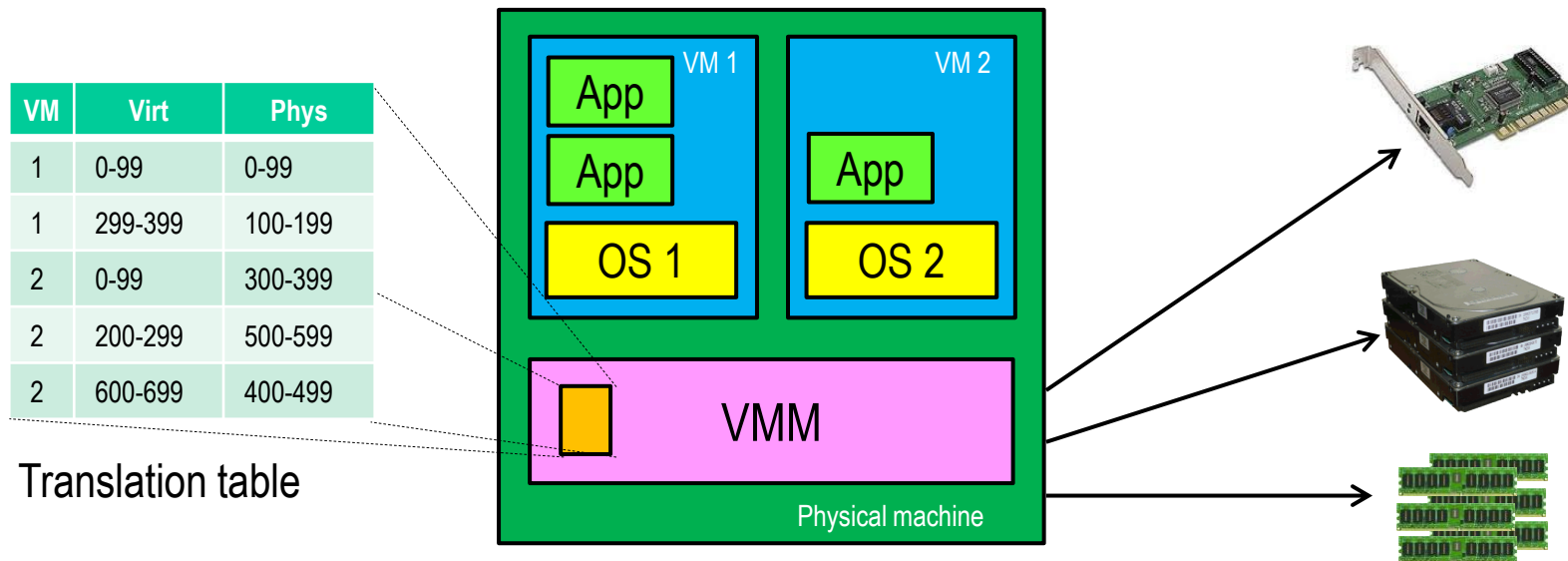
- Not-transparent virtualization
- Thin VMM
- Expose software interface to the virtual machine that is slightly modified from the host.
- Guest OS need to be modified.
- Simply transfer the execution of instructions which were hard to virtualized, directly to the host.

Para-Virtualization

- Privileged instructions of guest OS is delivered to the hypervisor by using hypercalls
- Hypercalls handles these instructions and accesses the h/w and return the result.
- Guest has authority to directly control of resources.



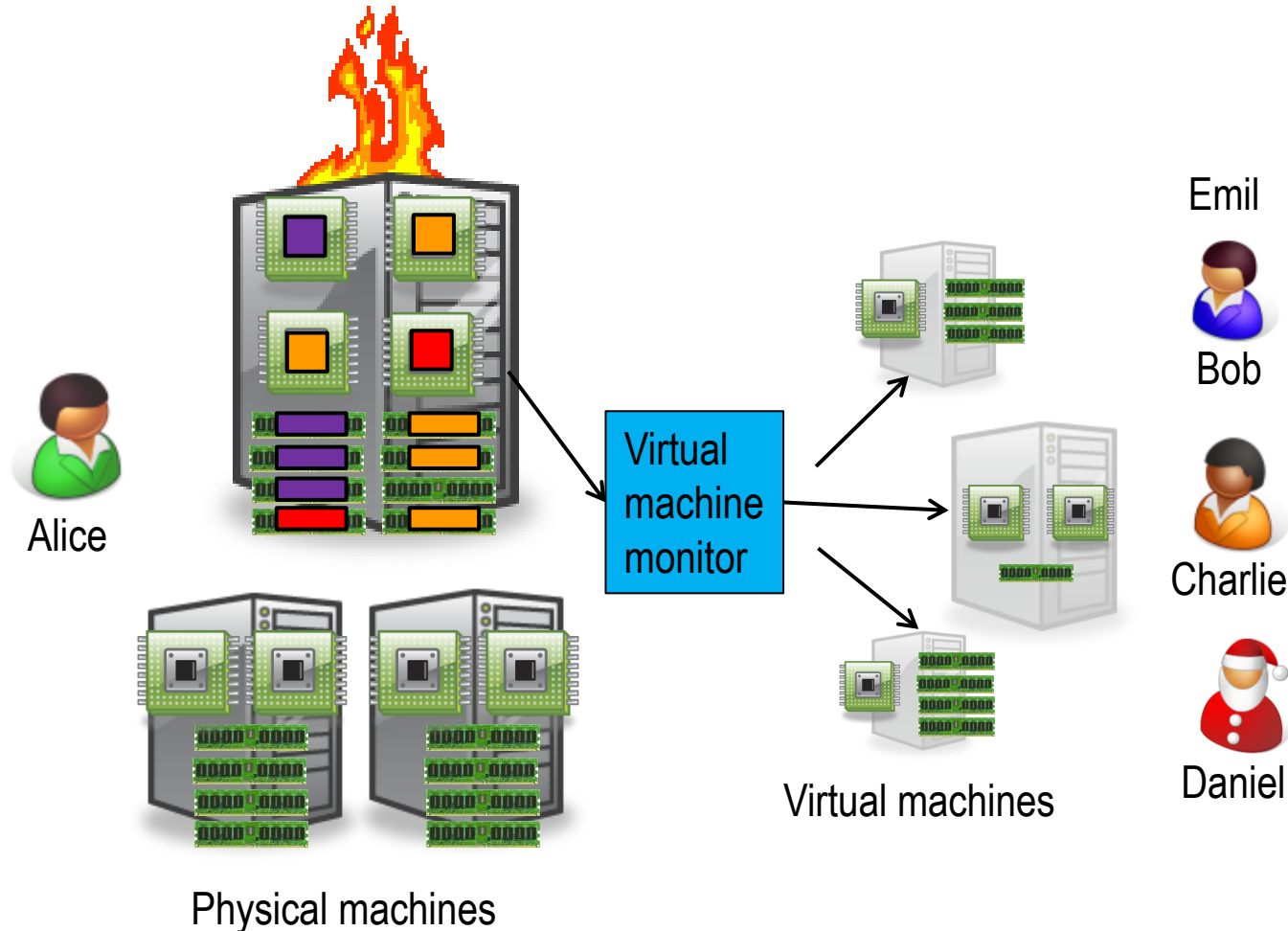
How does it work?



Resources (CPU, memory, ...) are virtualized

- VMM ("Hypervisor") has translation tables that map requests for virtual resources to physical resources
- Example: VM 1 accesses memory cell #323; VMM maps this to memory cell 123.
- For which resources does this (not) work?
- How do VMMs differ from OS kernels?

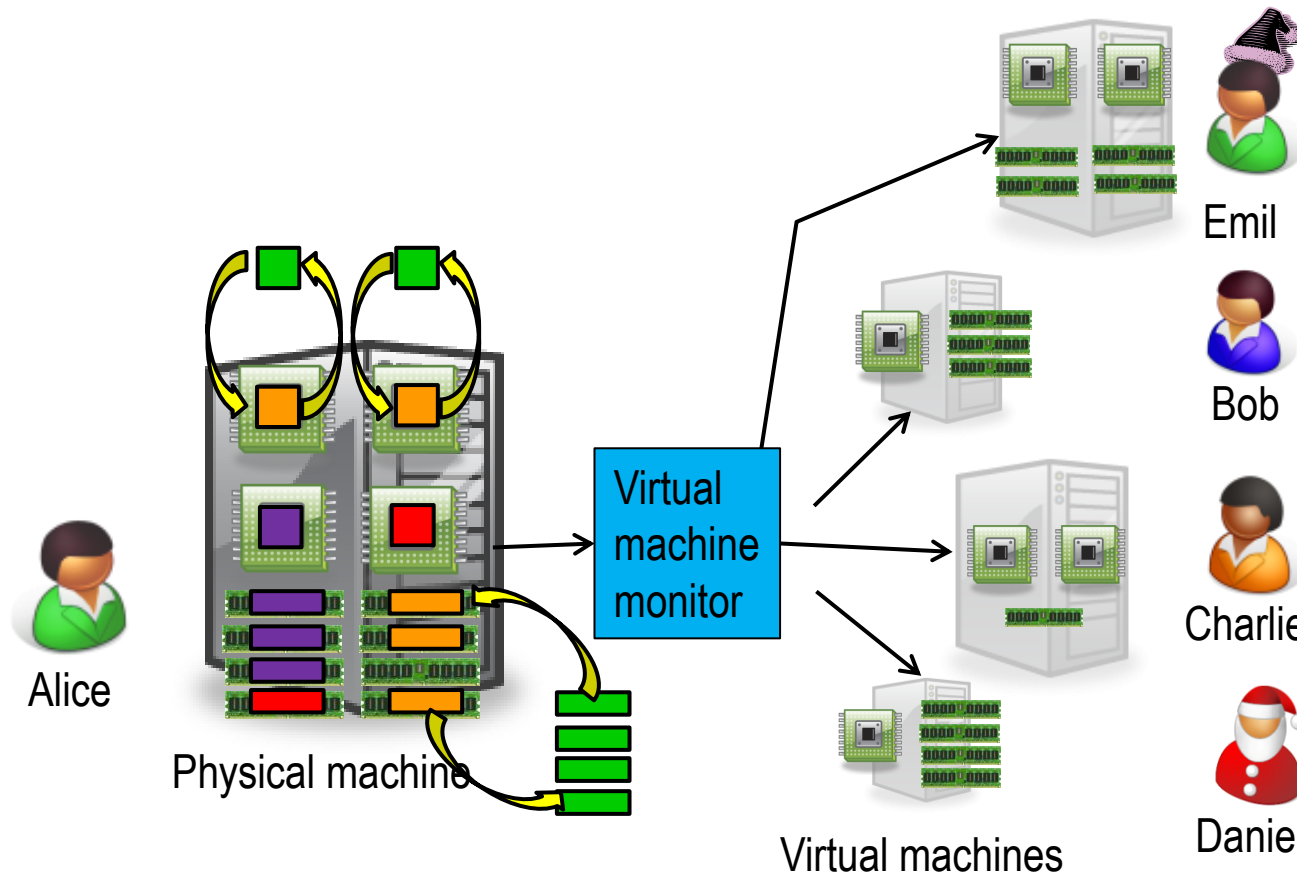
Benefit: Migration



What if the machine needs to be shut down?

- e.g., for maintenance, consolidation, ...
- Alice can **migrate** the VMs to different physical machines without any customers noticing

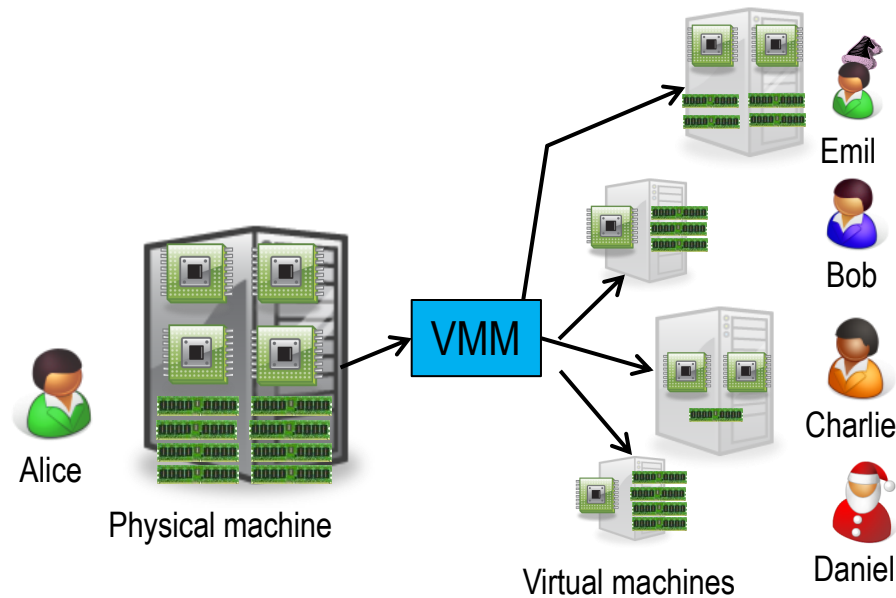
Benefit: Time sharing



What if Alice gets another customer?

- Multiple VMs can **time-share** the existing resources
- Result: Alice has more virtual CPUs and virtual memory than physical resources (but not all can be active at the same time)

Benefit and challenge: Isolation



Good: Emil can't access Charlie's data

Bad: What if the load suddenly increases?

- Example: Emil's VM shares CPUs with Charlie's VM, and Charlie suddenly starts a large compute job
- Emil's performance may decrease as a result
- VMM can move Emil's software to a different CPU, or migrate it to a different machine

Recap: Virtualization in the cloud

Gives cloud provider a lot of flexibility

- Can produce VMs with different capabilities
- Can migrate VMs if necessary (e.g., for maintenance)
- Can increase load by overcommitting resources

Provides security and isolation

- Programs in one VM cannot influence programs in another

Convenient for users

- Complete control over the virtual 'hardware' (can install own operating system, own applications, ...)

But: Performance may be hard to predict

- Load changes in other VMs on the same physical machine may affect the performance seen by the customer

Thank You!